

Configuration Manual

Praneeth Varma Danthuluri

1 Pre-requisite

1.1 Hardware Requirements

1. Computing Power:
 - A computer with sufficient processing power, ideally with a multi-core processor.
 - Adequate RAM, with a minimum recommended size (e.g., 8GB or more).
 - If deep learning models are used, a GPU with good computing capability (like NVIDIA CUDA-enabled GPU) is beneficial.
2. Storage Space:
 - Sufficient hard disk space for storing the dataset, software, and models. The size may vary based on the dataset and models used.

1.2 Software Dependencies

1. Operating System:
 - A stable operating system like Windows, macOS, or a Linux distribution.
2. Programming Language and Frameworks:
 - Python (with a recommended version, e.g., Python 3.8 or above).
 - Machine Learning libraries like TensorFlow, PyTorch (for MLP Regressor, if applicable), and Scikit-learn (for RandomForest and Linear Regression models).
 - Libraries for data analysis and visualization, such as Pandas, NumPy, Matplotlib, and Seaborn.
3. Development Tools:
 - Integrated Development Environment (IDE) such as PyCharm, Jupyter Notebook, or Visual Studio Code.
 - Version control system like Git.
4. Web Development Tools (for the web application):
 - Streamlit for building the web interface.

- Web server software, if applicable.
5. Data Preprocessing Tools:
 - Libraries for data preprocessing (e.g., Scikit-learn, Pandas).

2 Software Dependencies

2.1 Setting Up the Development Environment

1. Choose a Development Environment:
 - Select an Integrated Development Environment (IDE) like Jupyter Notebook, PyCharm, or Visual Studio Code.
2. Install Python:
 - Ensure Python (version 3.8 or later is recommended) is installed on your system. You can download it from python.org.
3. Setting Up Jupyter Notebook:
 - If you don't have Jupyter Notebook installed, you can install it via pip:
`pip install notebook`
 - To launch Jupyter Notebook:
`jupyter notebook`

2.2 Installing Required Libraries and Dependencies

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
```

3 Setup and Installation

3.1 Source and Format of the Data

- **Data Source:** The dataset used in this project is a CSV file named 'Test_data.csv'.
- **Data Format:** The data is structured in a comma-separated values (CSV) format, which is a common format for storing tabular data.

3.2 Code Snippet for Loading the Dataset

1. Import Pandas Library:

Ensure that Pandas is imported to handle data loading and manipulation:

```
import pandas as pd
```

2. Specify File Path:

Define the file path where your dataset is located. If it's in the same directory as your notebook, only the file name is needed:

```
file_path = 'Test_data.csv'
```

3. Load the Dataset:

Use Pandas to read the CSV record and cargo it right into a DataFrame:

```
data = pd.read_csv(file_path)
```

4. Verify Data Loading:

It's an amazing practice to verify that the statistics have been loaded successfully. you can view the primary few rows of the DataFrame the use of:

```
data.head()
```

```
[2]: # Load the dataset
file_path = 'Test_data.csv'
data = pd.read_csv(file_path)

[3]: data.head()
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_count	dst_host_srv_count	dst_host_same_srv_rate	d
0	0	tcp	private	REJ	0	0	0	0	0	0	...	255	10	0.04	
1	0	tcp	private	REJ	0	0	0	0	0	0	...	255	1	0.00	
2	2	tcp	ftp_data	SF	12983	0	0	0	0	0	...	134	86	0.61	
3	0	icmp	eco_i	SF	20	0	0	0	0	0	...	3	57	1.00	
4	1	tcp	telnet	RSTO	0	15	0	0	0	0	...	29	86	0.31	

5 rows × 41 columns

4 Data Preprocessing

Steps for Data Cleaning and Transformation

1. Import Necessary Libraries:

- Import libraries like Pandas, Matplotlib, Seaborn, and Scikit-learn's preprocessing and model selection tools:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
from sklearn.compose import ColumnTransformer
```

2. Preprocessing Steps

- Create a column transformer for preprocessing numerical and express facts:

```
preprocessor = ColumnTransformer(
```

```
    transformers=[
```

```
        ('num', StandardScaler(), numerical_cols),
```

```
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
```

```
    ])
```

```
[12]: # Create a column transformer for preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ])

```

Feature Engineering and Selection Methods

- specific feature engineering and selection strategies used on your project may be detailed here. This could encompass steps like encoding categorical variables, feature scaling, or choosing capabilities primarily based on their importance scores.

5 Exploratory Data Analysis (EDA)

Techniques Used for Data Visualization and Analysis

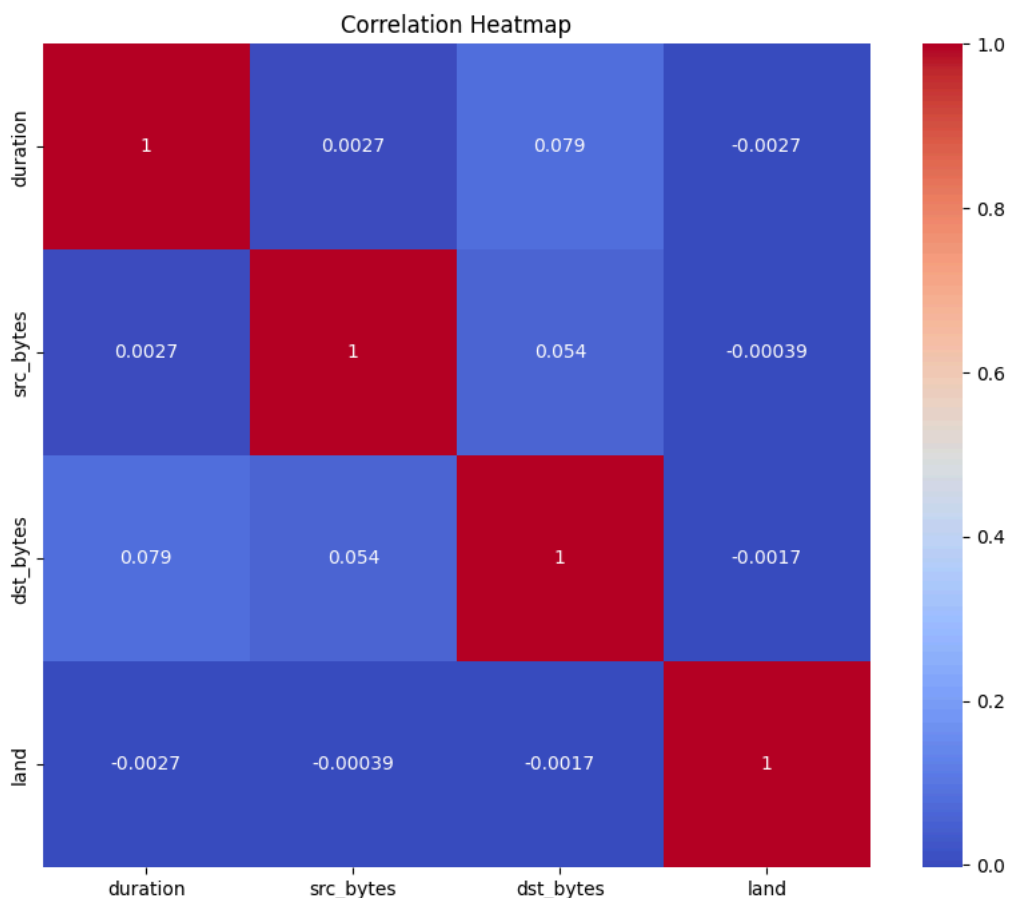
1. Initial Data Inspection:

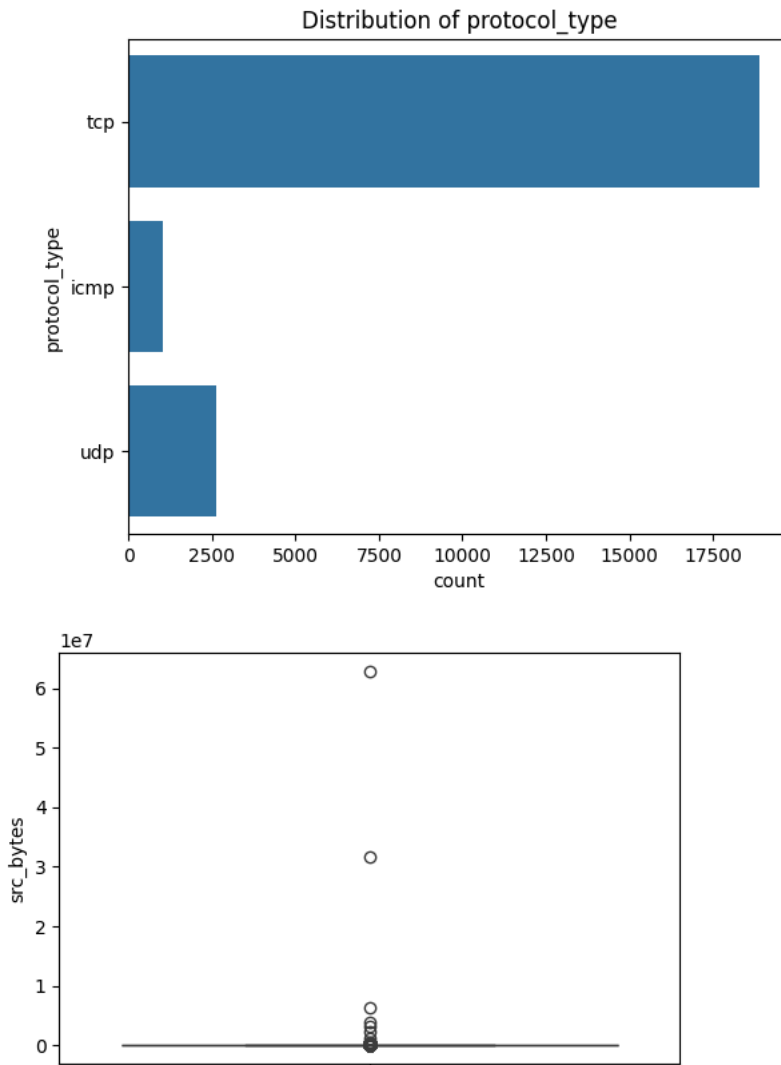
- Use `data.head()` to view the primary few rows of the dataset:

`data.head()`

2. Additional EDA Techniques:

- encompass other EDA techniques and visualizations like scatter plots, correlation matrices, etc., primarily based on the code and methods used for your notebook.





Insights Derived from EDA

- Summarize the important thing insights obtained out of your EDA, consisting of relationships between variables, data distributions, and any anomalies or outliers diagnosed.

6 Model Development and Training

Selection of Machine Learning Algorithms

Algorithm Choice:

- The project makes use of a Multi-Layer Perceptron (MLP) Regressor, which is a kind of neural community.

Method of model training, inclusive of Parameter Tuning

1. Data Preparation:

- Preprocess the input features (X) and target variable (y):

```
X = data.drop('src_bytes', axis=1) # Example feature column
y = data['src_bytes'] # Example target column
```

```

X_processed = preprocessor.fit_transform(X)
    })

[13]: # Apply preprocessing
X = data.drop('src_bytes', axis=1)
y = data['src_bytes']
X_processed = preprocessor.fit_transform(X)

```

2. Defining the Model:

- Instantiate the MLPRegressor with specified parameters:

```

from sklearn.neural_network import MLPRegressor

mlp_model = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=300,
random_state=1)

```

```

: from sklearn.neural_network import MLPRegressor

# Defining the MLPRegressor model
mlp_model = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=300, random_state=1)

# Training the model
mlp_model.fit(X_train, y_train)

# Scoring the model on the test set
mlp_score = mlp_model.score(X_test, y_test)
print(f"MLPRegressor Test Score: {mlp_score:.4f}")

```

3. Training the Model:

- Fit the model to the training data:

```

mlp_model.fit(X_train, y_train)

```

7 Model Evaluation and Validation

7.1 Techniques for Model Evaluation

1. Scoring the Model:

- Evaluate the model on the test set using the scoring method:

```

mlp_score = mlp_model.score(X_test, y_test)

print(f"MLPRegressor Test Score: {mlp_score:.4f}")

```

```

[15]: from sklearn.neural_network import MLPRegressor

# Defining the MLPRegressor model
mlp_model = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=300, random_state=1)

# Training the model
mlp_model.fit(X_train, y_train)

# Scoring the model on the test set
mlp_score = mlp_model.score(X_test, y_test)
print(f"MLPRegressor Test Score: {mlp_score:.4f}")

MLPRegressor Test Score: 0.0391

```

7.2 Metrics Used for Validation

1. Mean Squared Error (MSE):

- Calculate the MSE for the predictions to measure the model's performance:

```
from sklearn.metrics import mean_squared_error

y_pred_mlp = mlp_model.predict(X_test)

mse_mlp = mean_squared_error(y_test, y_pred_mlp)

print(f'Mean Squared Error (MLPRegressor): {mse_mlp}')
```

```
[16]: from sklearn.metrics import mean_squared_error

# Predictions and Evaluation
y_pred_mlp = mlp_model.predict(X_test)
mse_mlp = mean_squared_error(y_test, y_pred_mlp)
print(f'Mean Squared Error (MLPRegressor): {mse_mlp}')

Mean Squared Error (MLPRegressor): 222616103671.3241
```

8 Model Deployment

8.1 Steps for Model Serialization and Deployment

1. Extract Feature Names and Serialize:

- Extract feature names for later use and serialize them along with the model:

```
import joblib
# Function to extract feature names
def get_feature_names(column_transformer):
    # ... function details ...
feature_names = get_feature_names(preprocessor)
# Save feature names and the model
joblib.dump(feature_names, 'feature_names.pkl')
joblib.dump(your_model, 'model_filename.pkl')
```

2. Setting Up Deployment Environment:

- Instructions for setting up the environment where the model will be deployed, like a server or cloud platform.

```
'flag']

[26]: # Save the preprocessor and the model
joblib.dump(preprocessor, 'preprocessor.pkl')
joblib.dump(rf_model, 'random_forest_model.pkl')

[26]: ['random_forest_model.pkl']
```

9 Web Application Development

Steps to Develop the Web Application Interface

1. Setting Up Streamlit:

- The application uses Streamlit for the front end. Begin by importing Streamlit and other necessary libraries:

```
import streamlit as st

import pandas as pd
```

```
import joblib
```

2. Loading Pre-trained Models and Preprocessors:

- Load the serialized models and preprocessors using joblib:

```
preprocessor = joblib.load('preprocessor.pkl')
```

```
model = joblib.load('random_forest_model.pkl')
```

```
['flag']
```

```
[26]: # Save the preprocessor and the model
      joblib.dump(preprocessor, 'preprocessor.pkl')
      joblib.dump(rf_model, 'random_forest_model.pkl')
```

```
[26]: ['random_forest_model.pkl']
```

3. Building the Streamlit Interface:

- Set up the Streamlit app title and interface elements. For example:

```
st.title('Network Intrusion Detection System')
```

```
# Additional Streamlit interface elements
```

4. Function to Identify Columns:

- Implement a function to categorize columns as either categorical or numerical:

```
def identify_columns(df):
```

```
network_app.py
```

```
import streamlit as st
import pandas as pd
import joblib

# Load pre-trained models and preprocessors
preprocessor = joblib.load('preprocessor.pkl')
model = joblib.load('random_forest_model.pkl')

# Streamlit app title
st.title('Network Intrusion Detection System')

# Function to identify categorical and numerical columns
def identify_columns(df):
    categorical_cols = df.select_dtypes(include=['object',
'category']).columns.tolist()
    numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
    return categorical_cols, numerical_cols

# Load the dataset
data = pd.read_csv('Test_data.csv')
categorical_cols, numerical_cols = identify_columns(data)
```



```

# Collecting user inputs
user_input = {}

for col in categorical_cols:
    options = data[col].unique().tolist()
    user_input[col] = st.selectbox(f'Select {col}', options)

for col in numerical_cols:
    user_input[col] = st.number_input(f'Enter {col}',
value=float(data[col].mean()))

# Predict button
if st.button('Predict'):
    # Preprocess and predict
    input_df = pd.DataFrame([user_input])
    input_processed = preprocessor.transform(input_df)
    prediction = model.predict(input_processed)

    # Display prediction
    st.subheader('Prediction')
    st.write(prediction)

# About section
st.sidebar.header("About")
st.sidebar.info("This is a demo app for network intrusion detection using
machine learning.")

```

References

Gandhi, P. (2020) Using joblib to speed up your python pipelines, Medium. Available at: <https://towardsdatascience.com/using-joblib-to-speed-up-your-python-pipelines-dd97440c653d> (Accessed: 14 December 2023).

Gulati, A.P. (2022) Different ways of loading data using Python, Analytics Vidhya. Available at: <https://www.analyticsvidhya.com/blog/2022/08/different-ways-of-loading-data-using-python/> (Accessed: 14 December 2023).

Mahadevan, M. (2023) Step-by-step exploratory data analysis (EDA) using Python, Analytics Vidhya. Available at: <https://www.analyticsvidhya.com/blog/2022/07/step-by-step-exploratory-data-analysis-eda-using-python/> (Accessed: 14 December 2023).

Yung, I. (2018) A beginner's guide to training and deploying machine learning models using Python, freeCodeCamp.org. Available at: <https://www.freecodecamp.org/news/a-beginners-guide-to-training-and-deploying-machine-learning-models-using-python-48a313502e5a> (Accessed: 14 December 2023).